1. Projeto inicial

História escolhida: H1, reservar um horário de monitoria pelo bot (prioridade Alta).

Visão geral

O protótipo implementa a conversa do bot de WhatsApp em Python puro, sem a ligação com o WhatsApp, pois isso exigiria uma chave de API da Meta e um tempo de implementação incompatível com o prazo da atividade. A conversa segue o formato das listas de opções do WhatsApp, como nos bots de agendamento de barbearia: em cada etapa o bot manda uma pergunta com uma lista, e o aluno marca uma das opções em vez de digitar. O bot recebe o telefone do aluno, a opção marcada (ou o texto, se ele digitar mesmo assim) e o momento em que a mensagem chegou, e devolve a próxima pergunta com a lista dela. Como o momento entra junto com a mensagem, os testes conseguem simular qualquer dia e hora sem depender do relógio do computador. Os dados ficam em memória e são montados no começo de cada execução. O protótipo usa só a biblioteca padrão do Python.

O que não faz parte da H1 fica marcado no código com um comentário dizendo o que seria implementado ali futuramente. É o caso do envio e recebimento pelo WhatsApp, do banco de dados, do aviso de cancelamento (H2), da redução do limite por falta (H3) e dos relatórios (H4 e H5).

Classes e responsabilidades

Modelos. Classes simples de dados, sem regra: campus, curso, disciplina (com a lista fixa de assuntos dela), monitor (com a escolha, feita no cadastro, de atender em grupo ou não e, se atender, quantos alunos cabem por horário), horário (disciplina, monitor, início, sala e se foi cancelado), aluno (identificado pelo telefone) e reserva (horário, aluno e assunto). A duração de 30 minutos é fixa. A mensagem do bot também é um modelo: o texto da pergunta e a lista de opções, cada opção com um código e o título que o aluno vê.

Repositório em memória. Guarda os campus, cursos, disciplinas, horários, alunos e reservas, e responde às consultas que a agenda precisa: os cursos de um campus, as disciplinas de um curso, os horários de uma disciplina e as reservas de um horário ou de um aluno. Mais tarde seria trocado por um banco sem mudar a agenda.

Agenda. Concentra as regras da reserva. Ela lista os horários que um aluno pode reservar numa disciplina para um assunto, num dado momento, aplicando os critérios 2 e 3: horário não cancelado, começando pelo menos 1 hora depois, livre se o monitor atende um aluno por vez, ou com o mesmo assunto e com vaga se o monitor atende em grupo. Também faz a reserva, conferindo tudo de novo (outro aluno pode ter reservado o horário enquanto a conversa estava parada) e o limite de 2 reservas por semana na disciplina (critério 5). Quando a reserva não pode ser feita, a agenda lança um erro próprio que diz o motivo: horário indisponível ou limite semanal atingido. O registro do aluno é criado na primeira reserva (critério 4).

Bot. Conduz a conversa. Guarda, para cada telefone, em que etapa o aluno está (campus, curso, disciplina, assunto ou horário) e o que ele já escolheu. Em cada etapa monta a pergunta com a lista de opções, confere se a resposta é uma das opções da lista atual e, se não for, repete a pergunta com a mesma lista (critério 1). Isso cobre o aluno que digita em vez de marcar e o que marca uma opção de uma lista antiga. Na última etapa pede a reserva à agenda e transforma o resultado em mensagem: a confirmação com disciplina, monitor, dia, horário e sala (critério 4) ou a explicação do erro (critério 5). O bot não decide nenhuma regra de reserva, só pergunta e repassa.

Dados de exemplo. Monta dados próximos dos reais para a execução manual: três campus da UTFPR, entre eles Apucarana, com dois ou três cursos cada; no curso de Engenharia de Computação de

Apucarana, as quatro disciplinas citadas no e-mail (Cálculo, Física, POO e Estruturas de Dados), com os assuntos informados no plano de ensino de cada disciplina no portal do aluno, dois ou três monitores (alguns atendem em grupo, outros não) e seus horários.

Programa principal. Simula o WhatsApp no terminal: mostra a pergunta do bot e a lista de opções, e a pessoa escolhe uma delas pelo número, como se tocasse na opção. É a única parte que lê o teclado e não tem regra nenhuma.

Testes de aceitação

Os testes ficam num arquivo por critério dentro da pasta de testes, com unittest. Cada teste monta seus próprios dados e chama o bot ou a agenda diretamente, sem teclado e sem relógio do computador.

Critério 1, perguntas em ordem e resposta inválida. Um teste percorre a conversa inteira e confere que as perguntas vêm na ordem campus, curso, disciplina, assunto e horário, sempre com uma lista de opções (sucesso). Outro responde com texto digitado e com uma opção que não está na lista atual e confere que o bot recusa e repete a mesma pergunta (falha).

Critério 2, horários oferecidos. Testes conferem que não aparecem o horário cancelado, o horário que começa em menos de 1 hora e o horário de outra disciplina (falha), e que aparece o horário que começa exatamente 1 hora depois (sucesso, no limite).

Critério 3, individual e grupo. Testes conferem que o horário já reservado de um monitor que atende um aluno por vez some, que o horário de um monitor que atende em grupo aparece para o mesmo assunto enquanto há vaga (sucesso), e que não aparece para outro assunto nem quando está lotado (falha).

Critério 4, confirmação. Um teste faz a reserva e confere que ela foi gravada, que o aluno foi registrado pelo telefone e que a mensagem traz disciplina, monitor, dia, horário e sala (sucesso). Outro simula um segundo aluno pegando o horário individual antes da confirmação e confere que a reserva é recusada (exceção).

Critério 5, limite semanal. Um teste confere que a terceira reserva na mesma disciplina e na mesma semana é recusada com a explicação do limite (falha). Outros conferem que a reserva é aceita numa semana diferente e numa disciplina diferente (sucesso).

Execução

Os testes rodam com o unittest da biblioteca padrão a partir da pasta do projeto, e o programa principal abre a simulação no terminal. Os comandos exatos ficam no relatório de execução.

2. Relatório de execução

Como executar

O protótipo usa Python 3.14 e só a biblioteca padrão (`dataclasses`, `datetime` e `unittest`), sem nada para instalar. O arquivo principal é `codigo/principal.py`. Ele abre um menu para entrar como aluno, monitor, professor ou coordenação. Como aluno, o terminal simula o WhatsApp: a pessoa digita o número da opção da lista do bot, e `sair` volta ao menu. Nas outras telas, `0` volta. Os comandos rodam a partir da pasta do projeto. No Windows o comando é `py`; em outros sistemas, `python3`.

1. Sistema: `py codigo/principal.py`
2. Testes: `py -m unittest discover -s codigo -v` (85 testes, 37 deles da H1)

Plano e realizado

Trabalho individual: todas as etapas foram feitas por Eduardo Martinho.

1. 16 a 18/10, bot de agendamento (H1). Histórias em 17/10; projeto inicial, testes e código em 18/10. Em 19/10 foi corrigido o programa principal, que tinha ficado sem a linha que monta os dados.
2. 19/10, cancelamento de horário e relatório do monitor. Documentação técnica, relatório e cancelamento com aviso aos alunos (H2) feitos em 19/10. O relatório de horas (H5) ficou para 21/10.
3. 20/10, assunto e atendimento em grupo, dúvidas para o professor. O assunto e o grupo entraram na H1 em 18/10. Os assuntos mais procurados (H4) foram feitos em 20/10.
4. 21/10, limite semanal e faltas. O limite de 2 reservas por semana entrou na H1 em 18/10, e presença e faltas (H3) foram antecipadas para 20/10. Em 21/10 foram feitos o relatório de horas (H5) e as telas do monitor, do professor e da coordenação.
5. 22/10, pendências: PDF final, arquivo .zip e autoavaliação, adiantados para 21/10.

Atrasos e mudanças

A H1 é a história implementada para a avaliação. As outras quatro também foram implementadas, cada uma com testes próprios, mas sem projeto inicial, porque o enunciado pede projeto só da história escolhida. O relatório de horas atrasou dois dias em relação ao plano.

A ligação com o WhatsApp ficou de fora porque exige uma chave da API da Meta e mais tempo do que havia. O terminal faz o papel do WhatsApp, e o código marca com comentários onde entrariam o envio das mensagens, o login e o banco de dados.

O bot não aceita texto livre: em cada etapa o aluno marca uma opção de uma lista, como nas listas do WhatsApp, e o bot não recebe respostas que não entende. Por isso o critério 1 da H1 trocou "opções numeradas" por "lista de opções para marcar". Também passou a ser o monitor quem escolhe, no cadastro, se atende em grupo, porque o e-mail diz que só alguns preferem. Os assuntos de cada disciplina são uma lista fixa, tirada dos planos de ensino de Engenharia de Computação de Apucarana, para dar para saber quando dois alunos têm a mesma dúvida.

Em 18/10 a escolha do horário virou duas etapas, primeiro o dia e depois o horário desse dia, porque a lista do WhatsApp aceita no máximo 10 opções e uma lista única com o semestre todo ficava confusa. A lista de dias mostra os 30 primeiros dias com vaga, e toda lista com mais de 10 opções mostra 9 e "Ver mais". O critério 1 da H1 passou a citar o dia, e os testes já commitados foram ajustados e ganharam 7 casos.

3. Documentação técnica

1. Histórias de usuário

O sistema agenda as monitorias da UTFPR, e o aluno usa tudo por um bot de WhatsApp. Os tipos de usuário são aluno, monitor, professor e coordenação. Cada atendimento dura 30 minutos. No cadastro, cada monitor escolhe se atende em grupo: quem não atende recebe um aluno por horário, e quem atende diz quantos alunos cabem em cada horário dele.

H1. Reservar um horário de monitoria pelo bot

Como aluno, eu quero escolher pelo bot o campus, o curso, a disciplina e o assunto da minha dúvida e reservar um horário livre para ser atendido sem precisar combinar com o monitor por mensagem.

Critérios de aceitação:

1. O bot pergunta campus, curso, disciplina, assunto, dia e horário, nessa ordem, sempre com uma lista de opções para marcar, como as listas do WhatsApp. Se a resposta não for uma das opções da lista, o bot recusa e repete a pergunta.
2. O bot só oferece horários da disciplina escolhida que não foram cancelados pelo monitor e que começam pelo menos 1 hora depois do momento da conversa.
3. Se o monitor atende um aluno por vez, o horário que já tem reserva não aparece. Se ele atende em grupo, o horário que já tem reserva só aparece para quem escolheu o mesmo assunto, e só enquanto houver vaga.
4. Quando o aluno escolhe o horário, a reserva fica gravada no telefone dele (o registro do aluno é criado na primeira reserva) e o bot confirma a disciplina, o monitor, o dia, o horário e a sala.
5. Se o aluno já tem 2 reservas na mesma disciplina na semana do horário (de segunda a domingo), ele não consegue reservar outra nessa semana e o bot explica o limite.

Prioridade: Alta. Resolve o problema principal do e-mail, que são horários lotados enquanto outros ficam vazios, e a coordenação diz que só a reserva funcionando já ajuda muito.

Trecho do e-mail: "A ideia é que o aluno escolha a disciplina, veja os horários livres e reserve um atendimento." e "Se a reserva de horários funcionar, já ajuda muito."

H2. Cancelar um horário e avisar quem reservou

Como monitor, eu quero cancelar um horário em que não vou poder atender para que ninguém mais reserve e quem já reservou fique sabendo antes de ir até a sala.

Critérios de aceitação:

1. Depois de cancelado, o horário não aparece mais no bot.
2. Cada aluno que tinha reserva nesse horário recebe uma mensagem no WhatsApp com a disciplina, o dia e o horário cancelados.
3. As reservas desse horário passam a constar como canceladas pelo monitor e não contam no limite semanal do aluno.
4. O monitor não consegue cancelar um horário que já começou.

Prioridade: Alta. É a outra reclamação que os alunos fazem no e-mail. Sem o aviso, o aluno que reservou pela H1 continua indo até a sala para descobrir que não vai ter atendimento.

Trecho do e-mail: "às vezes precisa cancelar por causa de prova. Os alunos reclamam que só descobrem o cancelamento quando chegam na sala."

H3. Registrar presença e falta

Como monitor, eu quero marcar presença ou falta em cada reserva para que quem falta sem justificativa perca reservas e os horários não fiquem perdidos.

Critérios de aceitação:

1. Só dá para marcar presença ou falta em reservas de horários que já começaram.
2. Depois da primeira falta sem justificativa, o aluno só pode reservar 1 horário por semana naquela disciplina.
3. Depois da segunda falta sem justificativa, o aluno não consegue mais reservar naquela disciplina até o fim do semestre, e o bot avisa o motivo quando ele tenta.
4. Falta marcada como justificada não diminui o limite.

Prioridade: Média. Ajuda com os horários perdidos, mas nem os monitores concordam com o bloqueio. Por isso a regra primeiro reduz o limite e só depois bloqueia, e pode entrar depois que a reserva já estiver em uso.

Trecho do e-mail: "Quando o aluno reserva e não aparece, o horário fica perdido. Alguns monitores sugeriram bloquear quem falta muito, mas outros acham isso exagerado."

H4. Ver os assuntos que geram mais dúvidas

Como professor, eu quero ver os assuntos com mais atendimentos nas minhas disciplinas em um período para reforçar esses conteúdos em aula.

Critérios de aceitação:

1. A lista mostra cada assunto e quantos atendimentos teve, do mais procurado para o menos procurado.
2. Só contam os atendimentos com presença marcada, entre a data inicial e a final escolhidas.
3. O professor só vê os dados das disciplinas pelas quais é responsável.

Prioridade: Baixa. É útil para as aulas, mas não resolve nada do agendamento, e só tem dados depois que a H1 e a H3 estiverem funcionando.

Trecho do e-mail: "Os professores das disciplinas gostariam de saber quais conteúdos geram mais dúvidas, para reforçar em aula."

H5. Relatório mensal de horas do monitor

Como coordenação do curso, eu quero receber o relatório mensal com as horas atendidas por cada monitor para calcular o pagamento da bolsa.

Critérios de aceitação:

1. Cada horário com pelo menos um aluno presente conta 30 minutos, uma vez só, mesmo quando é em grupo.
2. Horários cancelados, ou em que todos os alunos faltaram, não contam.
3. O relatório é de um mês escolhido e mostra, para cada monitor, o total de horas e a lista dos atendimentos.
4. O monitor só consegue gerar o próprio relatório, a coordenação consegue gerar o de todos, e o bot nunca mostra horas nem valores de bolsa.

Prioridade: Média. A bolsa depende dele, mas o relatório é mensal e hoje já é feito de outro jeito. Também depende da reserva e da presença, então vem depois delas.

Trecho do e-mail: "A coordenação precisa de um relatório mensal com as horas atendidas por cada monitor, porque isso conta para o pagamento da bolsa. Essas informações de bolsa não podem ficar visíveis para todo mundo."

2. Testes de aceitação

História implementada: H1, reservar um horário de monitoria pelo bot. Os testes usam `unittest` e ficam em `codigo/testes/`, um arquivo por critério. Rodam com `py -m unittest discover -s codigo -v` a partir da pasta do projeto, e os 37 passam. Cada teste monta os próprios dados e passa o momento da conversa como parâmetro, então nenhum depende do teclado nem do relógio do computador.

Ao lado de cada teste está o tipo de cenário: sucesso, quando a ação deve dar certo; falha, quando o sistema deve recusar ou esconder alguma coisa; exceção, quando a agenda lança um erro.

Critério 1. Perguntas na ordem, sempre com lista, e recusa do que não está na lista

Arquivo `test_criterio_1_perguntas.py`.

1. `test_pergunta_campus_curso_disciplina_assunto_dia_e_horario_nessa_ordem` (sucesso)
2. `test_texto_digitado_e_recusado_e_a_pergunta_se_repete` (falha)
3. `test_opcao_de_uma_lista_antiga_e_recusada_e_a_pergunta_se_repete` (falha)
4. `test_depois_da_recusa_a_conversa_continua_da_mesma_etapa` (falha)
5. `test_depois_da_confirmacao_a_proxima_mensagem_recomeca_pelo_campus` (sucesso)
6. `test_mais_de_10_dias_mostra_9_e_a_opcao_ver_mais_dias` (sucesso)
7. `test_ate_10_dias_aparecem_todos_sem_ver_mais` (sucesso)
8. `test_lista_do_dia_mostra_todos_os_horarios_livres_desse_dia` (sucesso)
9. `test_curso_sem_monitoria_avisa_e_volta_para_a_lista_de_cursos` (falha)
10. `test_assunto_sem_horario_livre_avisa_e_volta_para_a_lista_de_assuntos` (falha)

Critério 2. Só horários da disciplina, não cancelados e com pelo menos 1 hora de antecedência

Arquivo `test_criterio_2_horarios_oferecidos.py`.

1. `test_horario_que_comeca_exatamente_1_hora_depois_aparece` (sucesso, no limite)
2. `test_horario_que_comeca_em_menos_de_1_hora_nao_aparece` (falha)
3. `test_horario_que_ja_passou_nao_aparece` (falha)
4. `test_horario_cancelado_pelo_monitor_nao_aparece` (falha)
5. `test_horario_de_outra_disciplina_nao_aparece` (falha)
6. `test_horarios_aparecem_em_ordem_de_inicio` (sucesso)
7. `test_bot_oferece_so_os_horarios_que_a_agenda_libera` (sucesso, com três horários que não podem aparecer)
8. `test_dia_so_com_horario_cancelado_nao_aparece` (falha)
9. `test_aparecem_so_os_30_primeiros_dias_com_vaga` (sucesso, no limite)

Critério 3. Horário individual some depois de reservado; em grupo, só para o mesmo assunto e com vaga

Arquivo `test_criterio_3_individual_e_grupo.py`.

1. `test_horario_individual_ja_reservado_nao_aparece` (falha)
2. `test_horario_individual_livre_aparece_para_qualquer_assunto` (sucesso)
3. `test_horario_em_grupo_vazio_aparece_para_qualquer_assunto` (sucesso)
4. `test_horario_em_grupo_com_vaga_aparece_para_o_mesmo_assunto` (sucesso)
5. `test_segundo_aluno_do_mesmo_assunto_consegue_reservar_o_horario_em_grupo` (sucesso)
6. `test_horario_em_grupo_nao_aparece_para_outro_assunto` (falha)
7. `test_horario_em_grupo_lotado_nao_aparece` (falha)

Critério 4. Reserva gravada no telefone, aluno registrado na primeira reserva e confirmação completa

Arquivo `test_criterio_4_confirmacao.py`.

1. `test_reserva_fica_gravada_no_telefone_do_aluno` (sucesso)
2. `test_aluno_e_registrado_pelo_telefone_na_primeira_reserva` (sucesso)
3. `test_confirmacao_traz_disciplina_monitor_dia_horario_e_sala` (sucesso)
4. `test_horario_individual_pego_por_outro_aluno_antes_da_confirmacao_e_recusado` (exceção, tratada pelo bot)
5. `test_agenda_lanca_erro_ao_reservar_horario_individual_ja_ocupado` (exceção)

Critério 5. No máximo 2 reservas por disciplina na semana do horário

Arquivo `test_criterio_5_limite_semanal.py`.

1. `test_terceira_reserva_na_mesma_disciplina_e_semana_e_recusada` (exceção)
2. `test_domingo_ainda_conta_na_mesma_semana` (exceção, no limite da semana)
3. `test_reserva_na_semana_seguinte_e_aceita` (sucesso)
4. `test_reserva_em_outra_disciplina_na_mesma_semana_e_aceita` (sucesso)
5. `test_conta_a_semana_do_horario_e_nao_a_da_conversa` (sucesso nas três primeiras reservas, exceção na quarta)
6. `test_bot_explica_o_limite_quando_a_reserva_e_recusada` (falha)

O que não foi coberto e por quê

O programa principal (`codigo/principal.py`) e as telas do monitor, do professor e da coordenação (`codigo/telas.py`) não têm teste automático, porque só leem o teclado e mostram o resultado no terminal, sem regra nenhuma. Foram testados à mão, percorrendo a conversa do bot até a confirmação e cada tela até o fim. Os dados de exemplo (`codigo/dados_exemplo.py`) também não têm teste: servem só para a execução manual, e os testes usam dados próprios e menores.

A ligação com o WhatsApp não existe no protótipo, então não há teste de envio e recebimento de mensagens, nem do limite de caracteres que o WhatsApp impõe ao título de cada opção. Também não há teste de duas mensagens chegando ao mesmo tempo: o protótipo roda num processo só, com os dados em memória. O caso parecido que existe, outro aluno pegar o horário enquanto a conversa estava parada, está coberto no critério 4.